

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in  
/usr/local/lib/python3.12/dist-packages (4.0.2)  
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in  
/usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

```
from pyspark.sql import SparkSession  
from pyspark.sql.functions import col, avg, count, sum as spark_sum,  
min as spark_min, max as spark_max, trim  
from pyspark.sql.types import TimestampType
```

```
# Create Spark Session
```

```
spark = SparkSession.builder \  
    .appName("Week 5 Spark Assignment") \  
    .getOrCreate()
```

```
# Sample data for assignment practice
```

```
data = [  
    (1, "2026-06-01", "West", "Electronics", 5000.0, "Jaipur", 22,  
    "Premium", "Active", "2026-06-01 10:30:00", "user1@gmail.com",  
    "user1", 5000.0, "S1"),  
    (1, "2026-06-01", "West", "Electronics", 5000.0, "Jaipur", 22,  
    "Premium", "Active", "2026-06-01 10:30:00", "user1@gmail.com",  
    "user1", 5000.0, "S1"),  
    (2, "2026-06-02", "West", "Clothing", 1500.0, "Jaipur", 25,  
    "Basic", None, "2026-06-02 12:00:00", "user2@gmail.com", "user2",  
    None, "S1"),  
    (3, "2026-06-03", "East", "Electronics", 7000.0, "Bikaner", 19,  
    "Premium", "Active", "2026-06-03 09:15:00", None, "user3", 7000.0,  
    "S2"),  
    (4, "2026-06-04", "South", "Grocery", 800.0, "Jodhpur", 31,  
    "Premium", None, "2026-06-04 11:45:00", "user4@gmail.com", "", 800.0,  
    "S2"),  
    (5, "2026-06-05", "West", "Grocery", 1200.0, "Jaipur", 28,  
    "Premium", "Active", "2026-06-05 14:20:00", "user5@gmail.com",  
    "user5", 1200.0, "S3"),  
]
```

```
columns = [  
    "user_id", "transaction_date", "region", "product_category",  
    "sale_amount",  
    "city", "age", "subscription", "status", "raw_timestamp",  
    "email", "username", "price", "store_id"  
]
```

```
df = spark.createDataFrame(data, columns)
```

```
# Extra data only to show city count greater than 100
```

```
extra_data = []
```

```

for i in range(6, 112):
    extra_data.append(
        (i, "2026-06-06", "North", "Grocery", 500.0, "Jaipur", 24,
         "Premium",
         "Active", "2026-06-06 10:00:00", f"user{i}@gmail.com",
         f"user{i}", 500.0, "S1")
    )

```

```

df_extra = spark.createDataFrame(extra_data, columns)
df = df.union(df_extra)

```

```

print("\nOriginal Data:")
df.show(10, truncate=False)

```

```

print("\nOriginal Schema:")
df.printSchema()

```

Original Data:

```

+-----+-----+-----+-----+-----+-----+-----+
+---+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
|user_id|transaction_date|region|product_category|sale_amount|city  |
age|subscription|status|raw_timestamp      |email          |username|
price |store_id|
+-----+-----+-----+-----+-----+-----+-----+
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+
|1      |2026-06-01      |West  |Electronics      |5000.0      |Jaipur |
22 |Premium      |Active|2026-06-01 10:30:00|user1@gmail.com|user1  |
5000.0|S1      |
|1      |2026-06-01      |West  |Electronics      |5000.0      |Jaipur |
22 |Premium      |Active|2026-06-01 10:30:00|user1@gmail.com|user1  |
5000.0|S1      |
|2      |2026-06-02      |West  |Clothing          |1500.0      |Jaipur |
25 |Basic        |NULL  |2026-06-02 12:00:00|user2@gmail.com|user2  |
NULL  |S1      |
|3      |2026-06-03      |East  |Electronics      |7000.0      |Bikaner|
19 |Premium      |Active|2026-06-03 09:15:00|NULL        |user3  |
7000.0|S2      |
|4      |2026-06-04      |South |Grocery          |800.0       |Jodhpur|
31 |Premium      |NULL  |2026-06-04 11:45:00|user4@gmail.com|        |
800.0 |S2      |
|5      |2026-06-05      |West  |Grocery          |1200.0      |Jaipur |
28 |Premium      |Active|2026-06-05 14:20:00|user5@gmail.com|user5  |
1200.0|S3      |
|6      |2026-06-06      |North |Grocery          |500.0       |Jaipur |
24 |Premium      |Active|2026-06-06 10:00:00|user6@gmail.com|user6  |
500.0 |S1      |
|7      |2026-06-06      |North |Grocery          |500.0       |Jaipur |

```

```

24 |Premium      |Active|2026-06-06 10:00:00|user7@gmail.com|user7  |
500.0 |S1          |
|8          |2026-06-06      |North |Grocery          |500.0      |Jaipur |
24 |Premium      |Active|2026-06-06 10:00:00|user8@gmail.com|user8  |
500.0 |S1          |
|9          |2026-06-06      |North |Grocery          |500.0      |Jaipur |
24 |Premium      |Active|2026-06-06 10:00:00|user9@gmail.com|user9  |
500.0 |S1          |
+-----+-----+-----+-----+-----+-----+
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+
only showing top 10 rows

```

Original Schema:

```

root
|-- user_id: long (nullable = true)
|-- transaction_date: string (nullable = true)
|-- region: string (nullable = true)
|-- product_category: string (nullable = true)
|-- sale_amount: double (nullable = true)
|-- city: string (nullable = true)
|-- age: long (nullable = true)
|-- subscription: string (nullable = true)
|-- status: string (nullable = true)
|-- raw_timestamp: string (nullable = true)
|-- email: string (nullable = true)
|-- username: string (nullable = true)
|-- price: double (nullable = true)
|-- store_id: string (nullable = true)

```

```

# Q3: Remove duplicate rows based on user_id and transaction_date
df_no_duplicates = df.dropDuplicates(["user_id", "transaction_date"])

print("\nAfter Removing Duplicates Based on user_id and
transaction_date:")
df_no_duplicates.show(10, truncate=False)

```

After Removing Duplicates Based on user_id and transaction_date:

```

+-----+-----+-----+-----+-----+-----+
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+
|user_id|transaction_date|region|product_category|sale_amount|city  |
age|subscription|status|raw_timestamp      |email          |username|
price |store_id|
+-----+-----+-----+-----+-----+-----+
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+
|1      |2026-06-01      |West  |Electronics      |5000.0      |Jaipur |

```

22	Premium	Active	2026-06-01 10:30:00	user1@gmail.com	user1	
5000.0	S1					
2		2026-06-02	West	Clothing	1500.0	Jaipur
25	Basic	NULL	2026-06-02 12:00:00	user2@gmail.com	user2	
NULL	S1					
3		2026-06-03	East	Electronics	7000.0	Bikaner
19	Premium	Active	2026-06-03 09:15:00	NULL	user3	
7000.0	S2					
4		2026-06-04	South	Grocery	800.0	Jodhpur
31	Premium	NULL	2026-06-04 11:45:00	user4@gmail.com		
800.0	S2					
5		2026-06-05	West	Grocery	1200.0	Jaipur
28	Premium	Active	2026-06-05 14:20:00	user5@gmail.com	user5	
1200.0	S3					
6		2026-06-06	North	Grocery	500.0	Jaipur
24	Premium	Active	2026-06-06 10:00:00	user6@gmail.com	user6	
500.0	S1					
7		2026-06-06	North	Grocery	500.0	Jaipur
24	Premium	Active	2026-06-06 10:00:00	user7@gmail.com	user7	
500.0	S1					
8		2026-06-06	North	Grocery	500.0	Jaipur
24	Premium	Active	2026-06-06 10:00:00	user8@gmail.com	user8	
500.0	S1					
9		2026-06-06	North	Grocery	500.0	Jaipur
24	Premium	Active	2026-06-06 10:00:00	user9@gmail.com	user9	
500.0	S1					
10		2026-06-06	North	Grocery	500.0	Jaipur
24	Premium	Active	2026-06-06 10:00:00	user10@gmail.com	user10	
500.0	S1					

```

+-----+-----+-----+-----+-----+-----+
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+
only showing top 10 rows

```

Q4: Filter region West and group by product_category to find average sale_amount

```

west_avg_sales = df.filter(col("region") == "West") \
    .groupBy("product_category") \
    .agg(avg("sale_amount").alias("average_sale_amount"))

```

```

print("\nAverage Sale Amount for West Region by Product Category:")
west_avg_sales.show()

```

Average Sale Amount for West Region by Product Category:

product_category	average_sale_amount
Electronics	5000.0

Clothing	1500.0
Grocery	1200.0

Q5: Fill null values in status column with Unknown

```
df_status_filled = df.na.fill({"status": "Unknown"})
```

```
print("\nAfter Filling Null Status with Unknown:")
```

```
df_status_filled.select("user_id", "status").show(10)
```

After Filling Null Status with Unknown:

user_id	status
1	Active
1	Active
2	Unknown
3	Active
4	Unknown
5	Active
6	Active
7	Active
8	Active
9	Active

only showing top 10 rows

Q6: Count records for each city where count is greater than 100

```
city_count = df.groupBy("city") \
    .agg(count("*").alias("total_records")) \
    .filter(col("total_records") > 100)
```

```
print("\nCities Having Record Count Greater Than 100:")
```

```
city_count.show()
```

Cities Having Record Count Greater Than 100:

city	total_records
Jaipur	110

Q8: Filter age between 18 and 30 and subscription Premium

```
premium_age_filter = df.filter(
    (col("age") >= 18) &
    (col("age") <= 30) &
    (col("subscription") == "Premium")
```

```
)
```

```
print("\nUsers with Age Between 18 and 30 and Premium Subscription:")
premium_age_filter.select("user_id", "age", "subscription").show(10)
```

Users with Age Between 18 and 30 and Premium Subscription:

user_id	age	subscription
1	22	Premium
1	22	Premium
3	19	Premium
5	28	Premium
6	24	Premium
7	24	Premium
8	24	Premium
9	24	Premium
10	24	Premium
11	24	Premium

only showing top 10 rows

```
# Q10: Cast raw_timestamp to TimestampType and rename it to event_time
df_timestamp = df.withColumn(
    "raw_timestamp",
    col("raw_timestamp").cast(TimestampType())
).withColumnRenamed("raw_timestamp", "event_time")
```

```
print("\nAfter Casting raw_timestamp to TimestampType and Renaming to event_time:")
df_timestamp.select("user_id", "event_time").show(10, truncate=False)
```

```
print("\nUpdated Schema:")
df_timestamp.printSchema()
```

After Casting raw_timestamp to TimestampType and Renaming to event_time:

user_id	event_time
1	2026-06-01 10:30:00
1	2026-06-01 10:30:00
2	2026-06-02 12:00:00
3	2026-06-03 09:15:00
4	2026-06-04 11:45:00
5	2026-06-05 14:20:00
6	2026-06-06 10:00:00
7	2026-06-06 10:00:00

```
|8      |2026-06-06 10:00:00|
|9      |2026-06-06 10:00:00|
+-----+-----+
only showing top 10 rows
```

Updated Schema:

```
root
|-- user_id: long (nullable = true)
|-- transaction_date: string (nullable = true)
|-- region: string (nullable = true)
|-- product_category: string (nullable = true)
|-- sale_amount: double (nullable = true)
|-- city: string (nullable = true)
|-- age: long (nullable = true)
|-- subscription: string (nullable = true)
|-- status: string (nullable = true)
|-- event_time: timestamp (nullable = true)
|-- email: string (nullable = true)
|-- username: string (nullable = true)
|-- price: double (nullable = true)
|-- store_id: string (nullable = true)
```

Q12: Remove rows where email is null OR username is empty

```
df_email_clean = df.filter(
    (col("email").isNotNull()) &
    (trim(col("username")) != "")
)
```

```
print("\nAfter Removing Rows with Null Email or Empty Username:")
df_email_clean.select("user_id", "email", "username").show(10,
truncate=False)
```

After Removing Rows with Null Email or Empty Username:

```
+-----+-----+-----+
|user_id|email          |username|
+-----+-----+-----+
|1      |user1@gmail.com|user1   |
|1      |user1@gmail.com|user1   |
|2      |user2@gmail.com|user2   |
|5      |user5@gmail.com|user5   |
|6      |user6@gmail.com|user6   |
|7      |user7@gmail.com|user7   |
|8      |user8@gmail.com|user8   |
|9      |user9@gmail.com|user9   |
|10     |user10@gmail.com|user10  |
|11     |user11@gmail.com|user11  |
+-----+-----+-----+
only showing top 10 rows
```

```
# Q13: Calculate min, max and mean of price column
price_stats = df.agg(
    spark_min("price").alias("minimum_price"),
    spark_max("price").alias("maximum_price"),
    avg("price").alias("mean_price")
)
```

```
print("\nPrice Statistics:")
price_stats.show()
```

Price Statistics:

minimum_price	maximum_price	mean_price
500.0	7000.0	648.6486486486486

```
# Q15: Final processing pipeline
# 1. Remove duplicates
# 2. Fill null prices with 0
# 3. Group by store_id and calculate total revenue
```

```
final_result = df.dropDuplicates() \
    .na.fill({"price": 0}) \
    .groupBy("store_id") \
    .agg(spark_sum("price").alias("total_revenue"))
```

```
print("\nFinal Processing Pipeline Result - Total Revenue by Store:")
final_result.show()
```

Final Processing Pipeline Result - Total Revenue by Store:

store_id	total_revenue
S3	1200.0
S2	7800.0
S1	58000.0